

Toil Job Store Storage — Cost & Architecture Analysis

Date: July 2026
Context: AWS ParallelCluster deployment with S3 VPC endpoint, dedicated 100 GB EBS job store, periodic cron zip backups to S3, and frequent `--restart` usage.

This document consolidates a cost and architecture assessment comparing:

- 1. Current setup (EBS file job store + S3 zip backups)
- 2. **JuiceFS mount** as `file:` job store (cluster-shared POSIX over S3)
- 3. Proposed prefix-based S3 job store (requires Toil code change — not implemented today)
- 4. Dedicated-bucket S3 job store (supported today, same cost as prefix-based)

Pricing basis: US East (N. Virginia) list prices, approximate. Adjust ~10–20% for other regions.

Executive summary

Question	Answer
Can Toil use an existing S3 bucket prefix today?	No — AWS job store uses a dedicated <code>{name}--toil</code> bucket per workflow store.
Does prefix vs dedicated bucket change S3 cost?	No — same \$/GB and same API request pricing.
Is S3 cheaper than 100 GB EBS for job store data?	Yes — ~\$8/mo (EBS gp3) vs ~\$2.30/mo (S3 Standard for 100 GB).
Main savings from moving primary store to S3?	Drop EBS charge + eliminate duplicate zip copies on S3.
Main tradeoff?	<code>--restart</code> latency: local EBS is faster than thousands of S3 LIST/GET calls, even via VPC endpoint.
Estimated annual savings (100 GB, single zip backup)?	~\$45–70/year if EBS is removed and duplicate backups are stopped.
Is JuiceFS viable for <code>file:</code> job store on Slurm?	Yes , if mounted on head and all workers — natural fit for ParallelCluster shared storage.
JuiceFS vs EBS for performance / stability?	Somewhat worse than local EBS; usually better than native S3 job store for <code>--restart</code> .
JuiceFS vs EBS + cron zip on cost?	Likely cheaper (drop EBS + redundant zips); data already on S3 via JuiceFS backend.

Current Toil AWS job store behavior

Locator format

```
aws:<region>:<name> → S3 bucket: <name>--toil
```

Example: `aws:us-west-2:my-workflow` → bucket `my-workflow--toil`.

The CLI help text “prefix” refers to a **name prefix for AWS resources (bucket names)**, not an S3 object key prefix.

What the implementation does

Operation	Behavior
<code>initialize()</code>	Creates a new S3 bucket; fails if bucket already exists
<code>resume()</code>	Requires bucket to exist
<code>destroy()</code>	Deletes the entire bucket and all contents
Object layout	Fixed keys at bucket root: <code>jobs/</code> , <code>files/</code> , <code>logs/</code> , <code>config.pickle</code> , etc.

Prefix-based job store (proposed, not implemented)

A future locator might look like:

```
aws:<region>:s3://org-bucket/toil/jobstores/run-abc123/
```

Implementation estimate: ~2–5 engineering days (~200–400 lines production code + tests). Most changes are localized to `AWSJobStore` init/resume/destroy and a new `delete_s3_prefix()` helper. Internal key prefixes (`jobs/`, `files/`, ...) can be prefixed at init time without touching every read/write path.

Deployment context (Augmet / ParallelCluster)

Component	Detail
Cluster	AWS ParallelCluster with Slurm
S3 access	S3 VPC endpoint (low latency, no internet egress for in-region S3)
Active job store	<code>file:</code> on dedicated always-mounted 100 GB EBS
Backup	Cron job zips job store → uploads to S3
Restart pattern	--restart used frequently due to unexpected failures
Shared paths (reference)	<code>/augmet-mp/job_stores/</code> , <code>/opt/augmet/augmet-engine-ro</code>
JuiceFS	Cluster-wide mounts available (S3-backed POSIX); alternative to dedicated EBS

AWS storage pricing reference (approximate)

Per GB-month (100 GB extrapolation)

Storage option	\$/GB-month	100 GB/month	Fit for active --restart?
EBS gp3	~\$0.08	~\$8.00	✓ Fast local reads
EBS gp2	~\$0.10	~\$10.00	✓ Fast
EBS snapshots	~\$0.05	~\$5.00	⚠ Restore step needed
S3 Standard	~\$0.023	~\$2.30	✓ Good for active store
S3 Standard-IA	~\$0.0125	~\$1.25*	⚠ Retrieval fees
S3 Glacier Instant Retrieval	~\$0.004	~\$0.40	⚠ Slower restart
S3 Glacier Flexible	~\$0.0036	~\$0.36	✗ Poor for frequent restart
S3 Glacier Deep Archive	~\$0.001	~\$0.10	✗ Hours to retrieve

*Plus per-GB retrieval charges on Standard-IA.

S3 API requests (same region, via Gateway VPC endpoint)

Operation	Approximate rate
GET / SELECT	~\$0.0004 per 1,000 requests
PUT / COPY / POST / LIST	~\$0.005 per 1,000 requests
Data transfer (in-region, Gateway endpoint)	\$0

Interface VPC endpoints incur hourly and per-GB processing charges; Gateway endpoints for S3 do not.

Annual storage-only comparison (100 GB)

Option	~\$/year
EBS gp3	~\$96
S3 Standard	~\$28
S3 Glacier Instant Retrieval	~\$5
S3 Glacier Deep Archive	~\$1

Cost models compared

Model A — Current (EBS primary + cron zip to S3)

```
Active workflow → file:/<ebs-mount>/jobstore
Restart         → local EBS (fast)
```

Backup → cron: zip → S3

Monthly cost components:

Line item	Typical cost
EBS gp3 100 GB (provisioned)	~\$8.00
S3 zip storage	~\$0.70 – \$14+ (see retention below)
S3 PUT (cron uploads)	~\$0.05
S3 GET during restart	\$0 (reads from EBS)
Total	~\$9 – \$22/month

Zip retention impact on S3 storage (assuming ~50 GB per compressed zip):

Retention	S3 storage cost
1 latest zip	~\$1.15/mo
6 monthly zips	~\$6.90/mo
12 monthly zips	~\$13.80/mo

Hidden cost: Data is often stored **twice** — full copy on EBS plus compressed copy (or copies) on S3.

Model B — Prefix-based S3 primary (proposed Toil feature)

Active workflow	→	aws:<region>:s3://bucket/toil/runs/<id>/
Restart	→	S3 via VPC endpoint (LIST + GET per job)
Backup	→	Optional (live data already on S3)
EBS	→	Removed

Monthly cost components:

Line item	Typical cost
EBS gp3 100 GB	\$0
S3 Standard live job store (~100 GB)	~\$2.30
S3 API (active run + ~30 restarts/mo)	~\$1 – 3 (up to ~\$5 at very high job churn)
Optional snapshot / second prefix	~\$0 – 2
Total	~\$3.30 – 5.30/month

Model C — Dedicated S3 bucket primary (supported today, no code change)

Same cost as Model B. Locator: `aws:<region>:<name>` → bucket `<name>--toil`.

Difference is operational only (many buckets vs one shared bucket with prefixes).

Model D — Worst case (S3 primary but keep EBS)

Adding S3 without removing EBS:

~\$12 – 14/month — pays for both; avoid unless transitioning.

Model E — JuiceFS `file`: job store (supported today, no Toil code change)

Active workflow

Restart

Backup

EBS

→

file:/<juicefs-mount>/job_stores/<run-id>/

→

POSIX reads via FUSE (metadata engine + S3 backend)

→

Optional (JuiceFS data already on S3)

→

Removed (if dedicated volume no longer needed)

JuiceFS presents a **cluster-shared POSIX filesystem** with object storage (typically S3) as the data plane and a separate metadata service (Redis, PostgreSQL, etc.).

Monthly cost components:

Line item	Typical cost
EBS gp3 100 GB (dedicated)	\$0 (if removed)
S3 storage (JuiceFS backend, ~100 GB)	~\$2.30
JuiceFS metadata engine	Variable (existing cluster infra; Redis/ElastiCache/RDS if dedicated)
S3 zip backups (cron)	\$0 (redundant — drop cron)
S3 API during active run	Lower than Model B (JuiceFS batches/coalesces); backend I/O absorbed by JuiceFS
Total (marginal)	~\$2.30/mo + metadata tier already running

Hidden savings: Eliminates duplicate storage (EBS full copy + cron zip on S3). JuiceFS stores one copy on S3.

When marginal cost is higher: If you provision a **new** metadata tier solely for job stores, factor in ElastiCache/RDS/EKS costs separately.

JuiceFS — performance, stability, and Toil fit

How Toil uses a file job store

`FileJobStore` requires a directory on a filesystem **shared by all worker nodes** (Slurm docs recommend a shared path for `--jobStore`).

Toil depends on:

Mechanism	Purpose
Atomic <code>rename</code>	Job updates: write <code>*.new</code> , then <code>os.rename()</code>
<code>os.path.exists</code> / directory walks	<code>--restart</code> scans <code>jobs/</code> , loads pickles
Many small files	Job descriptions, file-store metadata, logs
Spin-wait retries (up to ~35 s)	Tolerate NFS-like directory listing delay across nodes

Relevant implementation notes:

- `FileJobStore._wait_for_file()` is tuned for **NFS directory cache coherence** (~30 s).
- `FileJobStore.load_job()` documents races on **non-POSIX-compliant** filesystems (stale visibility → possible duplicate work).
- `FileJobStore.default_caching()` returns **False** — shared filesystems + Toil caching is known to be flaky; keep caching off on JuiceFS.
- Slurm docs warn that jobs may appear finished **before filesystem changes are visible** on other nodes; JuiceFS is in the same risk class as NFS.
- `toil.worker` prefers lock/coordination dirs **not on NFS** — use local `--coordinationDir` on each node.

Three-way comparison (performance & stability)

Dimension	Dedicated 100 GB EBS	JuiceFS mount	Native aws : S3 job store
--restart speed	Fastest (local block I/O)	Medium (FUSE + metadata + S3)	Slowest (LIST/GET per object)
Stability	High on attached node; poor Slurm fit if workers can't see volume	Medium — metadata engine + FUSE mounts	High durability; API/throttle edge cases
All-node Slurm access	EBS is normally single-node unless exported	✓ If cluster-wide mount	✓ IAM + VPC endpoint
Many small files	Good	Weaker — metadata-heavy	Many S3 API calls
Cross-node visibility	N/A (local)	Seconds-scale delays possible (like NFS)	Strong S3 consistency
Failure modes	Volume full, detach	Metadata outage, FUSE hang, stale handles	Throttling, SSE-C on shared bucket

Dimension	Dedicated 100 GB EBS	JuiceFS mount	Native aws : S3 job store
Slurm mount recovery	Less relevant (local)	Relevant — slurm_mount_recovery.py handles storage I/O errors on shared mounts	N/A

Restart latency ordering (typical)

```
Local EBS (head-attached) > JuiceFS (with node cache) > JuiceFS (cold)
>> aws: S3 job store
```

If EBS is **head-only** today, JuiceFS may **improve** worker-side file access while making head restart slightly slower than pure local EBS.

Performance — will it suffer vs EBS?

Yes, somewhat, for frequent **--restart**:

- 1. Restart is **metadata-heavy** — enumerate **jobs/**, open thousands of pickles; JuiceFS adds metadata round-trips and S3 reads.
- 2. Active runs issue constant small writes/deletes; FUSE + metadata can bottleneck on very large job counts (10k+).
- 3. Enable **JuiceFS client read cache** on compute nodes to reduce restart latency.

Stability — will it suffer vs EBS?

Somewhat yes — different failure modes:

Risk	EBS	JuiceFS
Mount / I/O errors	Rare on attached volume	FUSE hang, metadata disconnect, stale handles
Metadata dependency	None	Redis/PostgreSQL/etc. — outage affects entire mount
Slurm “job done, file not visible”	Uncommon on local disk	Same class as NFS — Toil retries, not a full guarantee
Duplicate job execution	Rare	Possible if rename/delete visibility is delayed
Concurrent job updates	Atomic rename on local FS	JuiceFS supports atomic rename; higher latency

Recommended Toil flags on JuiceFS

```
--jobStore file: /<juicefs-mount>/job_stores/<run-id>
--coordinationDir /local/ssd/toil-coord # local disk per node, NOT
```

```
JuiceFS
--batchLogsDir /<juicefs-mount>/toil-logs      # shared is fine
--caching false                               # FileJobStore default; keep
off on shared FS
--workDir /local/ssd/toil-work                # if nodes have local SSD
```

Use a **dedicated subdirectory per workflow** under the JuiceFS mount.

When to choose JuiceFS over EBS or S3

Prefer JuiceFS	Prefer dedicated EBS	Prefer native S3 job store
JuiceFS already on all nodes	Restart speed is paramount	Lowest storage \$ matters most
Want POSIX + drop EBS volume	Cannot tolerate metadata/FUSE outages	Willing to accept slower --restart
Frequent --restart, faster than S3	EBS is truly shared or head-only workflow	Prefix/bucket lifecycle without POSIX
Drop redundant cron zips	Huge job graphs; metadata already loaded	No FUSE/metadata tier to operate

Side-by-side monthly totals

Scenario 1 — Single latest zip on S3 (common)

Cost line	Model A (current)	Model B/C (S3 primary)	Model E (JuiceFS)
EBS 100 GB	\$8.00	\$0	\$0
S3 storage	~\$1.15 (zip)	~\$2.30 (live)	~\$2.30 (JuiceFS backend)
S3 requests / cron	~\$0.05	~\$1 – 3	~\$0 (no zip cron)
Metadata tier	\$0	\$0	\$0*
Total	~\$9.20	~\$3.30 – 5.30	~\$2.30
Savings vs A	—	~\$4 – 6/mo	~\$7/mo

*Assumes JuiceFS metadata engine already exists for the cluster; add ElastiCache/RDS costs if provisioning new infrastructure solely for job stores.

Scenario 2 — 12 months zip retention

Cost line	Model A (current)	Model B/C (S3 primary)	Model E (JuiceFS)
EBS 100 GB	\$8.00	\$0	\$0
S3 storage	~\$13.80	~\$2.30	~\$2.30
S3 requests	~\$0.10	~\$1 – 3	~\$0

Cost line	Model A (current)	Model B/C (S3 primary)	Model E (JuiceFS)
Total	~\$21.90	~\$3.30 – 5.30	~\$2.30
Savings vs A	—	~\$16 – 18/mo	~\$19/mo

Annual view (Scenario 1)

Model	~\$/year
Current (EBS + 1 zip)	~\$110
S3 primary (prefix or dedicated bucket)	~\$40 – 65
JuiceFS primary (drop EBS + zip)	~\$28
Annual savings (JuiceFS vs current)	~\$80

All models at a glance (100 GB, Scenario 1)

Model	~\$/mo	Restart speed	Slurm all-node	Toil code change
A — EBS + zip	~\$9	✔ Fastest	⚠ EBS often head-only	None
B/C — S3 primary	~\$3 – 5	⚠ Slowest	✔	Prefix: yes; bucket: none
E — JuiceFS	~\$2 – 3	✔ Medium	✔	None
D — S3 + EBS (avoid)	~\$12 – 14	Mixed	Mixed	—

Scale projection (live job store size)

Live size	EBS gp3/mo	S3 Standard/mo	Monthly delta
100 GB	~\$8.00	~\$2.30	~\$5.70
250 GB	~\$20.00	~\$5.75	~\$14.25
500 GB	~\$40.00	~\$11.50	~\$28.50
1 TB	~\$80.00	~\$23.00	~\$57.00

EBS is provisioned capacity; S3 bills actual usage (job store can grow beyond 100 GB without pre-provisioning).

S3 request cost estimate for frequent --restart

One restart on a workflow with ~5,000–10,000 jobs (order of magnitude):

Operation	Approx count	Cost
LIST jobs/ (paginated)	5 – 20	< \$0.001
GET job pickles	5,000 – 10,000	~\$0.002 – 0.004

Operation	Approx count	Cost
GET config / shared files	~10	negligible
Per restart		< \$0.01

~30 restarts/month → ~\$0.15 – 0.30/mo for restart reads alone.

Active workflow PUT/DELETE/LIST during normal execution: typically \$1 – 3/mo, occasionally up to ~\$5/mo at very high job churn. Unlikely to offset EBS savings at 100 GB scale.

Non-cost tradeoffs

Restart latency

	EBS file: job store	JuiceFS file: job store	S3 aws: job store
Read path	Local filesystem	FUSE → metadata → S3	S3 LIST/GET over VPC endpoint
Latency	Milliseconds per file	Milliseconds–tens of ms; cache helps	Tens of ms per object
Large job graphs	Fast directory scan	Metadata-heavy; many small reads	Paginated LIST + per-job GET
Fit for frequent -- restart	✔ Preferred	✔ Good compromise	⚠ Acceptable but slower

Operational

Topic	EBS + zip	JuiceFS	S3 primary
Duplicate data	Yes (EBS + S3 zip)	Single copy on S3 backend	Single copy
Cron / zip CPU	Required	Not needed	Optional
Bucket sprawl	N/A (file store)	N/A (uses JuiceFS bucket)	Many {name}--toil buckets unless prefix feature ships
Lifecycle / expiry	Manual zip rotation	S3 lifecycle on JuiceFS bucket	S3 lifecycle on toil/* prefix
IAM	Mount + S3 for backup	Mount + existing JuiceFS IAM	Workers need S3 read/write
Encryption (SSE-C)	N/A on file store	Inherited from JuiceFS/S3 config	Shared bucket may lack SSE-C

Topic	EBS + zip	JuiceFS	S3 primary
Metadata / FUSE ops	None	Redis/DB + client mounts	None
Slurm shared-FS quirks	N/A if local	Same as NFS (visibility delay)	N/A

Prefix vs dedicated bucket (cost-neutral)

	Dedicated bucket	Prefix in shared bucket
Storage \$	Same	Same
Request \$	Same	Same
VPC endpoint	Same	Same
Bucket limit (1000 default)	One bucket per job store	One bucket for many stores
Lifecycle policies	Per bucket	One rule for <code>prefix/*</code>
<code>toil clean</code>	Deletes whole bucket	Must delete prefix only (needs code)

Recommendations

If lowest AWS cost is the priority

- 1. Move primary job store to **S3** (dedicated bucket today; prefix when implemented) **or JuiceFS** (if already deployed).
- 2. **Remove** the dedicated 100 GB EBS volume if unused elsewhere.
- 3. **Stop or reduce** cron zip backups (avoid paying for data twice; especially redundant with JuiceFS).
- 4. Apply **S3 lifecycle** on the JuiceFS backend bucket or job-store prefix for completed runs.

If fastest `--restart` is the priority

- 1. **Keep EBS** (or fastest available local/shared FS) for the active job store.
- 2. **Reduce S3 backup cost:** keep one latest zip, or `aws s3 sync` instead of full zip history.
- 3. Optionally sync to S3 prefix on success for provenance without duplicating 12 months of zips.

If balancing cost, restart speed, and Slurm fit (recommended for this cluster)

JuiceFS as primary `file: job store` is often the best compromise:

- 1. Use `file:<juicefs-mount>/job_stores/<run-id>` on **all nodes**.
- 2. Remove dedicated 100 GB EBS and **stop cron zips**.
- 3. Set `--coordinationDir` to **local disk** on each node (not JuiceFS).
- 4. Keep `--caching false` on shared filesystem.
- 5. Enable JuiceFS **client read cache** on compute nodes if restarts are frequent.

Hybrid (balance)

During unstable run → file: on JuiceFS or EBS (fast restart)
After stable checkpoint → optional sync to S3 prefix for audit
Provenance / audit → S3 lifecycle on JuiceFS backend bucket; no separate zip pipeline

Decision matrix

Your priority	Choose
Lowest \$ + already have JuiceFS	Model E (JuiceFS)
Lowest \$ + no JuiceFS	Model B/C (S3 job store)
Fastest --restart	Model A EBS (if truly local/shared)
Fast restart + all-node Slurm + drop EBS	Model E (JuiceFS)
Minimal ops / fewest moving parts	Model B/C (S3) — no FUSE/metadata

Implementation backlog (prefix-based S3 job store)

If pursuing Model B with a shared bucket:

Area	Effort	Notes
Locator parsing	Small	e.g. <code>aws:<region>:s3://bucket/prefix/</code> ; keep legacy <code>aws:<region>:name</code>
<code>initialize()</code>	Medium	Bucket must exist; prefix must be empty
<code>resume()</code>	Small	Check <code>config.pickle</code> under prefix
<code>destroy()</code>	Medium	<code>delete_s3_prefix()</code> — handle versioned objects
Region handling	Small	<code>get_bucket_region()</code> for existing buckets
Tests + docs	Medium	New test class; fix misleading <code>JOBSTORE_HELP</code> text
Total	~2–5 days	

Risks: SSE-C on shared buckets, public URL ACLs vs Block Public Access, versioned bucket deletes, region mismatch.

What does *not* save money

- Prefix-based vs dedicated-bucket S3 layout (same pricing).
- Moving provenance to S3 if you **keep** EBS and **keep** full zip retention (triple-ish redundancy).
- Glacier tiers for job stores that need **frequent --restart**.
- JuiceFS if you **provision a new metadata tier** only for job stores (ElastiCache/RDS adds fixed cost).

- JuiceFS **and** cron zip **and** EBS simultaneously (triple redundancy).

Open inputs for refined estimates

Fill in for a tighter assessment:

Parameter	Your value
AWS region	
Live job store size (GB)	100 (assumed)
Compressed zip size (GB)	
Zip retention count	
Restarts per month	
Typical job count at restart	
VPC endpoint type (Gateway vs Interface)	Gateway (assumed)
JuiceFS mount path	
JuiceFS metadata engine (Redis/RDS/other)	
JuiceFS client cache enabled on compute nodes?	
EBS attached to head only or all nodes?	

Related code references

File	Relevance
src/toil/jobStores/aws/jobStore.py	AWS job store; bucket-only design
src/toil/jobStores/aws/jobStore.py — parse_jobstore_identifier()	<code>{name}</code> → <code>{name}-toil</code> bucket
src/toil/jobStores/fileJobStore.py	File job store; NFS retry logic, atomic rename, <code>default_caching()</code>
src/toil/options/common.py — JOBSTORE_HELP	Locator documentation
src/toil/lib/aws/s3.py — create_s3_bucket(), delete_s3_bucket()	Bucket lifecycle
docs/running/hpcEnvironments.rst	Slurm shared filesystem guidance
src/toil/batchSystems/slurm_mount_recovery.py	Storage I/O failure detection on shared mounts
src/toil/worker.py	Coordination dir should not be on NFS-like FS

File	Relevance
AGENTS.md	Slurm / <code>/augmet-mp/job_stores/</code> deployment notes

Revision history

Date	Notes
2026-07-02	Initial document from architecture and cost analysis session
2026-07-02	Added JuiceFS (Model E) — performance, stability, cost, and Toil tuning